

Trading Bot

A semi-autonomous, multi-asset algorithmic trading system for Alpaca (paper) — equities, crypto, and options-wheel — with an automated strategy lab, multi-source intel gates, and a self-supervising daemon.

Generated: 2026-04-29 · **Repo:** /Users/bharathkandala/Trading · **Branch:** main · **Python:** 3.11+

At a glance

Component	Count / Detail
Source modules (src/trading_bot/)	78 modules + 4 subpackages (roles, options, backtest, dashboard)
Roles (cooperating agents)	23 (daemon, supervisor, lab tiers)
Trading workflows	3 (stocks momentum, crypto momentum 24/7, options wheel)
Per-trade intel gates	6 named (Tier 1: earnings, fear/greed, reddit-spike; Tier 2: insider, macro shock, coingecko)
External data providers	10 (Polygon, Finnhub, Alpaca, FRED, GDELT, ApeWisdom, Alternative.me, CoinGecko, SEC EDGAR, Truist)
Alembic migrations	12
Test files	100+ pytest modules under tests/
Strategy templates (live)	MomentumStrategy (active), MeanReversionStrategy (disabled), Wheel CSP/CC
LLM integration	Anthropic Claude — Opus 4.7 for Strategy Architect, Haiku 4.5 for Tone Analyst

Document map

- **1. Project overview & architecture** — what the bot does, how the daemon, supervisor and lab cooperate.
- **2. The three workflows** — stocks momentum, crypto momentum, options-wheel — full entry/exit decision trees.
- **3. News & intel sources** — every external feed, each per-trade gate, thresholds and caches.
- **4. The Lab** — Optuna search, walk-forward, fitness, calibration drift, LLM-driven strategy generation, AST + sandbox validation, promotion.
- **5. Strategies & algorithms** — exact rules and math for momentum, mean reversion, wheel CSP/CC, regime detection, backtest harness.
- **6. Risk & portfolio** — risk manager gates, regime allocations, sector caps, position protection.
- **7. State, persistence, reporting** — state.db schema, 12 migrations, multi-email reporting cadence.
- **8. Schedule timetable & roles** — every cron, every role, what feeds what.
- **9. Configuration reference** — the full strategy/config.yaml annotated.

1. Project overview & architecture

1.1 What this bot is

This repository is a **semi-autonomous algorithmic trading bot** targeting an **Alpaca paper account**, with a hard, hardcoded paper-only check on the wheel strategy. It trades three asset classes side-by-side in a single coordinated process: US equities, USD-quoted crypto pairs, and listed equity options run as a covered-call wheel.

The system is structured as cooperating **roles** — discrete single-purpose agents that each implement a uniform *Role Protocol* (see `src/trading_bot/roles/base.py`). Roles run inside one of three host processes: the **daemon** (execution loop, scheduling, market-data and trading), the **supervisor** (drawdown sentinel, stall watchdog, schedule auditor) and the **lab** (overnight strategy evolution and parameter search).

1.2 The three host processes

Process	Entry point	What it does	Cadence
Daemon	<code>src/trading_bot/daemon.py:432</code>	APScheduler driving every trading workflow, intel warm-calls, repos (long/short) reconciliation	1000 Hz
Supervisor	<code>src/trading_bot/supervisor.py</code>	Independent watchdog process. Reads the daemon heartbeat. If stall (> 120 s) emits a critical email.	60 Hz
Lab	<code>src/trading_bot/lab.py:95</code>	Overnight strategy evolution. Optuna parameter search at 02:00 ET (Backlog note: Scheduled)	1 Hz

1.3 Boot sequence

- Daemon reads `data/paper_active.json` (active config), overridable via `TRADING_BOT_CONFIG`.
- Alembic migrations run automatically against `data/state.db` at startup (`daemon.py:438-456`).
- WAL mode is enabled at engine creation (`journal_mode=WAL`, `synchronous=NORMAL`) so daemon, lab and supervisor read concurrently without locking.
- Heartbeat written via `tmp+rename`, with `PID + uuid` suffix in the `tmp` filename to prevent collisions on concurrent scheduler jobs (`state_heartbeat.py:17`).
- All scheduled jobs are wrapped in `_wrap()` which checks the `data/pause.flag` sentinel before running — the `AccountSentinel` writes that flag on drawdown breach.

1.4 Stall and drawdown safety

The system has two independent watchdogs. The **StallDetector** (`watchdog_stall.py:22-52`) compares `data/heartbeat.json` mtime to wall clock; if older than 120 s it issues a **single** `launchctl kickstart -k gui/{uid}/{daemon_label}` and emits a critical email. The **AccountSentinel** (`roles/account_sentinel.py`) reconciles the Alpaca account against an equity high-water mark (`state_hwm.py`); on breach it writes `data/pause.flag`, which freezes every order-placing job until cleared.

Pause semantics: `data/pause.flag` blocks `intel_scan` and `crypto_scan` (the two order-placing workflows). It does *not* block the daily digest, schedule auditor, alert drainer or reconciler, which remain safe to run.

2. The three trading workflows

All three workflows share a uniform pipeline: **preflight** (idempotency, position checks) → **signal** (technical evaluation) → **intel gates** (news / sentiment vetoes) → **risk gate** (sizing, caps, regime allocation) → **order**. Gates can only veto entries; nothing in the pipeline opens a position that wasn't first signalled. This makes adding a new gate *strictly safer*, never more aggressive.

2.1 Workflow 1 — Equity Momentum (stocks)

- **Purpose:** hourly intraday scan of US equities for short-term momentum entries.
- **Runner:** StockScannerRole — roles/stock_scanner.py.
- **Cadence:** hourly cron, weekdays 09:00–15:00 ET (registered as intel_scan in scheduler_jobs.py).
- **Universe:** two-stage screener — stage 1 ranks the entire Polygon US universe, stage 2 evaluates the top-100 across three lanes.

Stage-1 ranking (screener.py:75-102)

```
score = one_day_return * 1.4
+ (stock_5d_return - SPY_5d_return)
+ min(volume_ratio_20d, 3.0) * 2.0
# top 100 carried into stage 2
```

Stage-2 lanes (strategy_lanes.py)

- **MomentumLane** (lines 41–86): $RSI(14) \in [55, 70]$, $MACD > \text{signal}$, $\text{close} > \text{EMA}(20)$, 5-day return > 0 . Conviction 0.4–0.9.
- **MeanReversionLane** (lines 89–124): $RSI < 30$, close below 20-day Bollinger lower band (2σ).
- **BreakoutLane** (lines 127–160): close above prior 20-day high, volume $> 1.5 \times$ 20-day average.

Entry decision (orchestrator.py:154-248)

- Has open position / pending order / already traded today → **skip**.
- Insufficient bars (< 26) or strategy returns HOLD → **skip**.
- News sentiment floor: if Polygon score $< \text{strategy.sentiment_floor}$ (default -0.5) → **skip**.
- Macro shock gate ($GDELT \leq -3.0$) → **skip**.
- Earnings gate (Finnhub): earnings within next 5 trading days → **skip**.
- Insider cluster gate (Finnhub, OFF by default): ≥ 5 sells in 90 d → **skip**.
- Risk manager: per-trade risk $\leq 1\%$, position $\leq 10\%$, concentration $\leq 5\%$, daily loss $> -2\%$, weekly $> -5\%$.
- If all gates pass: place **buy** order; $\text{stop} = \max(\text{EMA}20, \text{entry} \times (1 - \text{stop_pct}))$.

Position sizing (strategy.py:86-93)

```
risk_qty = floor((equity * risk_pct/100) / per_share_risk)
concentration_qty = floor((equity * max_concentration_pct/100) / entry)
qty = min(risk_qty, concentration_qty)
if qty < 1: HOLD with reason 'qty < 1 share'
```

2.2 Workflow 2 — Crypto Momentum (24/7)

- **Purpose:** continuous scan of ~30 USD-quoted crypto pairs.
- **Runner:** `CryptoScannerRole` (`roles/crypto_scanner.py`) — runs every 30 minutes, 24×7.
- **Universe discovery** (`crypto_universe.py`): pulls Alpaca active crypto assets, filters to USD-quoted, excludes stablecoins (USDC, USDT, DAI...), applies operator blocklist `strategy/crypto_blocklist.yaml`.
- **Signal:** shares the *MomentumLane* evaluator with equities — same RSI/MACD/EMA rules.

Crypto-specific intel gates

- **Polygon news sentiment floor is bypassed** — Polygon news is equity-focused.
- **Macro shock gate** ($GDELT \leq -3.0$): blocks — the only macro signal that affects crypto entries.
- **Crypto Fear & Greed gate** (Alternative.me): index outside **[20, 80]** → skip.
- **Reddit spike gate** (ApeWisdom `r/CryptoCurrency`): mentions $\geq 2.0 \times$ 24h-ago → skip.
- **CoinGecko sentiment gate** (OFF by default): community sentiment < 50% → skip when enabled.
- **Asset class cap:** crypto allocation capped at 25% in trending up, 5% in risk off (regime-dependent).

Crypto runs 24/7 and ignores RTH. The same heartbeat / pause flag still apply, so a daemon stall halts crypto trading the moment the supervisor writes `pause.flag` — even at 3 AM.

2.3 Workflow 3 — Covered-Call Wheel (options)

- **Purpose:** generate yield via covered call premium on eligible equities.
- **Single state machine** per symbol: `CSP_OPEN` → `ASSIGNED` → `CC_OPEN` → `CLOSED` (`options/wheel_state.py:17-22`).
- **Paper-only:** hard-coded URL prefix check. Enabled 2026-04-29.

Wheel runners (`options/wheel_runner.py`)

Job	Time (ET)	What it does
<code>wheel_universe_build</code>	21:30 nightly	Walk Alpaca <code>optionable_us_equities</code> (~6,000), filter via Finnhub (market cap, listing age). Ca
<code>iv_capture</code>	09:45 weekdays	Capture ATM 30 d IV per eligible name; persist to <code>option_iv_history</code> .
<code>wheel_scan</code>	10:15 weekdays	Single CSP/CC entry pass. Preflight, chain pick, sector cap, journal.
<code>wheel_manage</code>	10:00‐15:00 every 30s	Take profit, DTE force-close, delta-breach roll, cycle close.

Preflight gates (`wheel_lane.py:62-89`)

- Cycle phase $\in \{\text{none}, \text{ASSIGNED}\}$ — `CSP_OPEN`/`CC_OPEN` are managed elsewhere.
- Regime $\in \{\text{trending_up}, \text{sideways}\}$; `trending_down` and `risk_off` blocked.
- $VIX \in [15.0, 30.0]$; below = premiums too thin, above = vol-of-vol risk.
- Sentiment ≥ -0.3 (looser than equity floor — we already screen by other means).
- IV rank $\geq 30\%$ over trailing 252 d.
- WSB spike check (ApeWisdom `r/WallStreetBets`): mentions $\geq 2.0 \times$ baseline → skip.

-
- Earnings window: skip if Finnhub flags earnings between today and (expiration + 2 d).

Contract picking (chain.py:42-77)

Both `pick_csp_contract` and `pick_cc_contract` filter contracts by $DTE \in [30, 45]$, $abs(delta) \in [0.20, 0.30]$, open interest ≥ 100 , spread $\leq \$0.10$ or 5%% of mid, bid $\geq \$0.20$. The CSP picker sells the put closest to delta 0.25; the CC picker additionally requires $strike \geq cost_basis$ so being called away locks in profit.

Management cycle (wheel_runner.py:282-435)

- **Take-profit:** if $mid \leq credit \times (1 - 0.50) \rightarrow$ buy-to-close.
- **DTE force-close:** if $dte \leq 21 \rightarrow$ close.
- **Delta breach roll:** $CSP > 0.45 / CC > 0.55$, roll up to **2 times** per cycle.
- **Roll mechanic:** buy-to-close current + sell-to-open one expiry out at same delta band; falls back to defensive close if no replacement contract.
- **Sector cap:** 25% per GICS sector (yfinance-backed cache, 14 d TTL); unknown sectors do not gate.

3. News & intel sources — per workflow

The bot ingests **10 external providers** across price data, news, sentiment and macro context. Six of them feed the **per-trade gates** added in commit f1df0f6 ('six per-trade news/intel gates'). Every gate has soft-fail semantics: a network error or 4xx returns *None* and lets the entry through, never blocks it. This protects against an upstream outage taking the bot offline.

3.1 Provider catalog

Provider	Endpoint / file	What it produces	Cache & TTL
Polygon (via Massive)	massive_client.py	Per-ticker news + Polygon-built sentiment label, 300+ terms, 24h lookback	SQLite news, 24h; sentiment, 10d; massive_grouped, 10d
Finnhub	intelligence_finnhub.py	Earnings calendar, insider transactions (Form 4)	in-process, 24 h
Alpaca News	intelligence.py:70-118	Per-symbol financial headlines (2 d lookback)	Bundled in IntelligenceAggregator
Alpaca Market Data	alpaca_client.py	Real-time quotes, bars, options chain, account state	Store SQLite (24 h)
FRED	intelligence.py:122-154	Macro series (VIX VIXCLS, DGS10, DFF, fed funds rate)	SQLite; regime detector consumes VIX
GDELT 2.0	intelligence.py:157-196	Macro tone score (-10..+10) from "stock market & US&P&C&S&S OR Federal Reserve" 24h window	SQLite&P&C&S&S
ApeWisdom	intelligence_apewisdom.py	Reddit r/CryptoCurrency and r/WallStreetBets mentions	in-process, not per scan
Alternative.me	intel_gates.py:182-219	Crypto Fear & Greed Index (0‐100)	1 h in-process
CoinGecko	intel_gates.py:322-377	Per-coin sentiment_votes_up_percentage	30 min in-process (OFF by default)
SEC EDGAR	intelligence.py:199-249	Recent Form 4 insider filings (RSS)	Bundled context only, no gate
Truth Social	vip_tweets.py	VIP handle posts; keyword severity scoring (HIGHER=more severe)	Cache, seen.json (persistent)

3.2 The six per-trade intel gates

Tier 1 — fast, always-on

Gate	Source	Rule	Default	Workflow
Stock Earnings	Finnhub /calendar/earnings	Skip if earnings in next N trading days	lookahead 5 d	Stocks, Wheel
Crypto Fear & Greed	Alternative.me	Skip if index outside [floor, ceiling]	N, [20, 80]	Crypto
Crypto Reddit Spike	ApeWisdom r/CryptoCurrency	Skip if mentions ≥ multiplier	1, 1245, 20	Crypto

Tier 2 — defensive, conditionally enabled

Gate	Source	Rule	Default	Workflow
Stock Insider Cluster	Finnhub /stock/insider-transactions	Skip if ≥ 5 insider sells in 90 d	OFF (noisy on 10b5D)	Stocks
Macro Shock	GDELT 2.0 doc API	Skip ALL entries if tone ≤ threshold	0, 1, −3	All
CoinGecko Sentiment	CoinGecko v3	Skip if sentiment_votes_up_pct < floor	OFF, floor 50%	Crypto

3.3 Per-workflow gate matrix

Gate / Filter	Stocks	Crypto	Wheel
Existing position / pending order / traded-today	yes	yes	yes (cycle phase)
Insufficient bars (< 26)	yes	yes	n/a (chain-driven)
Strategy signal HOLD	yes	yes	yes (preflight)
Polygon news sentiment floor	yes (-0.5)	no (bypass)	yes (-0.3, looser)
Macro shock (GDELT)	yes	yes	yes
Earnings gate (Finnhub)	yes (5 d)	no	yes (today — expiration+2)
Insider cluster (Finnhub)	off by default	no	off
Crypto Fear & Greed	no	yes [20, 80]	no
Reddit r/CryptoCurrency spike	no	yes (×2.0)	no
WSB r/WallStreetBets spike	no	no	yes (×2.0)
CoinGecko community sentiment	no	off by default	no
Regime allowed	any	any	trending_up, sideways
VIX band	n/a	n/a	[15, 30]
IV rank floor	n/a	n/a	≥ 30 over 252 d
Sector cap	n/a	n/a	25% per sector
Risk manager (sizing, caps)	yes	yes	yes (collateral)

3.4 VIP listener — alert-only, never auto-trades

VipListenerRole polls Truth Social RSS for configured handles (currently only Donald Trump in `strategy/vip_handles.yaml`) every 30 minutes during US market hours. The keyword severity scorer in `vip_tweets.py` tags posts as HIGH on tokens like *tariff*, *sanction*, *fed chair*, *executive order*, *\$spy*, *\$btc*, and MED on macro words like *inflation*, *recession*, *jobs report*, *deficit*. **This system never auto-trades, never auto-halts and never auto-vetoes** — it emits a critical email and writes the post id to `data/vip_seen.json` so it doesn't re-fire on the next poll.

4. The Lab — automated strategy evolution

The Lab is a separate launchd-managed process whose job is to find better strategy parameters than what the daemon is currently running, validate them across multiple time slices, and either flag promising candidates or auto-promote them into the live config. It runs four scheduled jobs per day.

4.1 Lab schedule

Time (ET)	Job	Role	Outcome
02:00 daily	param_search	ParamOptimizerRole	100 Optuna TPE trials & leaderboard rows + 1 EvolutionRun summary
02:45 daily	auto_promote	PromoterRole	Read top leaderboard, evaluate gates, possibly write paper_active.json
05:00 daily	calibrate	CalibratorRole	Spearman drift score between predictions and 30-trade realized P&L &
06:00 Saturday	saturday_evolve	StrategyArchitect & CodeRefiner	CodeRefiner generates 1–3 templates & AST + sandbox validation

4.2 Parameter search — Optuna TPE

ParamOptimizerRole (roles/param_optimizer.py:41-135) builds an `optuna.create_study` with `TPESampler` in *maximize* mode and runs `n_trials=100` by default. The search space lives in `param_space.py`; only the *momentum* template is currently enumerated:

```
PARAM_SPACE['momentum'] = {
    'rsi_lower': (50, 60, float),
    'rsi_upper': (65, 75, float),
    'ema_period': (15, 30, int),
    'stop_pct': (3.0, 7.0, float), # percent, not fraction
    'sentiment_floor': (-1.0, 0.0, float),
}
```

- Each trial calls `BacktestEngineerRole.safe_run` for a 6-fold walk-forward backtest, computes a fitness score, writes a leaderboard row.
- Trials whose backtest fails are converted to `optuna.TrialPruned()` — pruned, not penalised.
- The lab universe is read from `strategy/opportunities.md` (top 25 stocks, curated by the Universe Curator at 07:30 ET); fallback baseline is SPY, QQQ, AAPL, MSFT, AMD, NVDA, GOOGL, META, JPM, JNJ.

4.3 Walk-forward mechanics

`walkforward.default_folds` (`walkforward.py:27-54`) builds **6 folds** with **train=12 months, test=3 months**, walking the cursor forward by the test window each fold — tiled, non-overlapping test slices. Before each fold, `_ensureBarsWarmed` backfills the `BarStore` from Alpaca (or the Polygon/Massive loader) for any missing symbols and drops symbols whose fetch fails.

4.4 Fitness function

```
fitness = alpha_vs_spy_x + 0.5 * sortino - 0.5 * dd_penalty
# dd_penalty = max(0, max_dd_pct - 20.0) # only above 20% drawdown

# Hard promotion gate (all three must hold):
alpha_vs_spy_x >= 1.5
```



```
sortino >= 1.0
max_dd_pct <= 20.0
```

- $\alpha_{vs_spy_x} = \text{strategy_return} / \text{SPY period return}$, clamped to 100.0 when SPY is flat.
- sortino annualised via $\sqrt{252}$ using only negative daily returns for the downside std.
- max_dd_pct from peak-tracking on equity curve; reported as worst-case **max** across folds.
- Aggregate alpha and sortino reported as **mean** across folds; drawdown reported as **max** (worst-case treatment).

4.5 Promotion — auto and manual

Auto (*PromoterRole* at 02:45 ET)

- Halt gate first: any active PromoterHalt row → skip with reason halted_by_calibrator.
- Hard threshold gate ($\alpha \geq 1.5x$, $\text{sortino} \geq 1.0$, $\text{dd} \leq 20\%$).
- **10% delta gate**: candidate fitness must beat incumbent by $\geq 10\%$ (`MIN_FITNESS_DELTA = 0.10`) — this stops nightly leaderboard noise from churning the active config.
- First-time promotion (no incumbent or incumbent fitness ≤ 0) auto-passes the delta gate.
- On approval: `promote_atomically` writes via `tmp+rename`, stamps `version='auto-YYYYMMDD-HHMMSS'`, `promoted_by='lab-promoter'`, sends a **Strategy Promotion email** with params + risk-caps diff.
- LabPromotionStore records the promotion for first-24 h validation: scans, entries, near-misses, validated_at.

Manual CLI (*promote_cli.py*)

- `bot promote paper` — replicates the auto logic on demand.
- `bot promote live` — **three independent gates**: (1) `ALPACA_LIVE_API_KEY/SECRET` set, (2) `--i-know-this-is-real-money` flag, (3) operator types literal 'YES, FLIP TO LIVE' exactly.
- Live promotion locks **stricter caps**: `max_position` 5% (vs paper 10%), `daily_loss` 1.5% (vs 2%), `max_drawdown` 10% (vs 20%) — halved across the board.
- Writes a `ConfigHistory` audit row.

4.6 Calibration — drift detection

CalibratorRole at 05:00 ET pairs the most-recent leaderboard's `per_trade_predictions_json` with realised P&L from `data/closed_trades.db` over a rolling 30-trade / 30-day window. The drift score is the **Spearman rank correlation** between predicted and realised per-trade returns, implemented in pure numpy in `calibration.py:13-31`.

Spearman corr	Severity	Action
> 0.5	ok	no action
$0.3 \leq \text{corr} \leq 0.5$	warning	log + email

Spearman corr	Severity	Action
< 0.3	high	insert PromoterHalt with halted_until = now + 7 days
n < 10 trades	insufficient_data	no action

4.7 LLM-driven strategy generation

On Saturdays at 06:00 ET, StrategyArchitectRole asks Claude to generate 1–3 net-new strategy templates conforming to a strict signature: `evaluate(symbol, ind, equity) -> Signal` and `from_params` classmethod. The system prompt enforces hard constraints in plain English.

- Models: `claude-opus-4-7` for the architect, `claude-haiku-4-5-20251001` for the tone analyst — overridable via `env`.
- Imports allowed: `pandas`, `numpy`, `ta`, `math`, `datetime`, `dataclasses`, `typing`, `decimal`, `enum`.
- Imports prohibited: `os`, `sys`, `subprocess`, `requests`, `urllib`, `eval`, `exec`, `__import__`, `open`.
- No I/O. No future bars. 5-year backtest must run in < 30 s.
- Output is strict JSON; parser strips ````json` fences and falls back to a regex search.
- Every API call goes through `anthropic_client.py` with retry, cost logging (`AnthropicCostLog` table), and a `CostHalt` budget cap.

4.8 Generated-code validation pipeline

CodeReviewerRole runs three deterministic checks on each pending `TemplateProposal`:

- **1. AST allowlist** (`ast_validator.py`:67-94): walks the AST, rejects non-allowlisted import roots, rejects calls to `FORBIDDEN_CALLS = {eval, exec, compile, __import__, open}`, catches `__builtins__.__import__` attribute trickery.
- **2. Sandbox runtime** (`sandbox_runner.py`): writes source + tests to a `tempdir`, runs `python -m pytest` in a subprocess with `walltime_s=30`, `mem_mb=512` via `RLIMIT_CPU` and `RLIMIT_AS`.
- **3. Disposition**: accepted modules land in `src/trading_bot/strategies/_evolved/<name>/`; rejects go to `_archive/`. `review_status` is updated; on accept, `accepted_at` is stamped.

5. Strategies & algorithms — concrete rules

5.1 MomentumStrategy (active in trending_up)

strategy.py:24-105. Long-only, integer-share quantities.

Param	Default	Lab range
rsi_lower	55	50–60
rsi_upper	70	65–75
ema_period	20	15–30
per_trade_risk_pct	0.5	fixed
stop_pct	0.05 (5%)	3%–7%
max_concentration_pct	4.5	fixed
sentiment_floor	−0.5	−1.0…0.0

Entry rules (all four must pass):

- $RSI(14) \in [rsi_lower, rsi_upper]$
- MACD line > MACD signal
- Last close > EMA(20)
- 5-day return > 0

Stop: $\max(EMA20, \text{entry} \times (1 - \text{stop_pct}))$ — whichever is higher (tighter).

5.2 MeanReversionStrategy (currently disabled)

strategy.py:108-165. $RSI < [25, 35]$, $\text{close} \leq 1.01 \times EMA20$, 5-day return $\geq -5\%$. Same sizing math as Momentum but with a flat stop_pct=4%. **Currently returns None for all regimes** — backtest profit factor of 0.66 over 49 trades led to disabling pending rework. Returning None means “no entries this regime” but existing positions are still managed by exits/stops.

5.3 Strategy router

Regime	Strategy returned	Notes
trending_up	MomentumStrategy	Active path
sideways	None	Mean-reversion disabled (PF 0.66)
trending_down	None	Insufficient backtest data
risk_off	None	Defensive — no new entries

5.4 Regime detection (regime.py)

Inputs: SPY 250-day bars + optional VIX (FRED VIXCLS). Logic in detect_regime_from_bars (regime.py:46-132):

-
- $VIX > 28 \rightarrow$ forced **risk_off** (highest priority).
 - 20-day annualised realised vol $> 22\%$ (Phase 0b change from 30%) OR 20-day drawdown $< -10\% \rightarrow$ **risk_off**.
 - $close > EMA50 > EMA200$, $vol < 25\%$, $VIX \leq 22 \rightarrow$ **trending_up**.
 - $close < EMA50 < EMA200 \rightarrow$ **trending_down**.
 - Otherwise $\rightarrow$ **sideways**.

5.5 Backtest harness (backtest/simulator.py)

Day-by-day replay that reuses the live entry strategy, regime detector, indicators, and risk manager *verbatim* — only the day clock, simulated portfolio, and exit-leg logic are new. This means a backtest result reflects what *would actually* happen in production.

- **Trading-day clock** from `BarStore.trading_dates(SPY)` — honors real holidays.
- **Daily loop**: resolve open positions $\rightarrow$ mark to market $\rightarrow$ record P&L $\rightarrow$ update streaks $\rightarrow$ detect regime (with VIX backfill if missing) $\rightarrow$ check halt (loss caps) $\rightarrow$ if not halted, attempt entries.
- **Exit resolution**: missing bar $\rightarrow$ time exit at `max_hold_days`; otherwise check stop first ($low \leq stop \rightarrow$ "stop"), then take-profit ($high \geq tp \rightarrow$ "tp"), then time. **Stop wins on intra-bar conflict** (conservative).
- **Optional trailing-stop logic** (off by default per empirical sweep): ratchet stop to break-even at $+3\%$ peak unrealised, then to $peak \times giveback_fraction$ at $+5\%$.
- **Entry execution at next-day open** avoids look-ahead bias. 2:1 R:R take-profit ($tp = entry + 2 \times risk_per_share$).
- Per-strategy, per-regime and per-asset-class metrics in `backtest/metrics.py`: win rate, profit factor, expectancy, Sharpe (annualised), max drawdown.

6. Risk & portfolio management

6.1 RiskManager.check — every trade gate

`risk_manager.py:26-51`. The single `check(order, account, positions, state, regime)` method is called by every workflow before submitting an order.

- **1. Halted flag:** any halt (manual or auto) → `RiskRuleViolation`.
- **2. Per-trade risk:** $(\text{entry} - \text{stop}) \times \text{qty} \leq 1\% \text{ account equity}$.
- **3. Max position:** $\text{notional} \leq 10\% \text{ account equity}$.
- **4. Concentration:** $\text{symbol notional} \leq 5\% \text{ account equity}$.
- **5. Asset-class caps:** regime-dependent (see table below).
- **6. Loss limits:** daily $\geq -2\%$, weekly $\geq -5\%$.

6.2 Regime-dependent allocation caps

Regime	Stocks	Crypto	Options	Cash floor
trending_up	60%	25%	15%	0%
sideways	40%	20%	20%	20%
trending_down	30%	15%	10%	45%
risk_off	10%	5%	0%	85%

6.3 Position protection (auto-stops)

`OrderStewardRole` runs every **:20 and :50** of every hour, 24/7 (`scheduler_jobs.py:78`) and calls `position_protection.py`. For every unprotected position it computes a strategy-aligned stop ($\max(\text{EMA20}, \text{close} \times (1 - \text{stop_pct}))$) and either places it (*protect*) or market-closes the position (*flatten*). Off-hours equity flattens are deferred — Alpaca rejects equity market orders outside RTH; crypto is protected 24/7. Failures are per-symbol so one bad name doesn't abort the sweep.

6.4 Strategy Coach & the Hold-SPY fallback

`StrategyCoachRole` runs at 06:00 ET daily and computes a 30-day realised alpha multiplier ($\text{strategy_return} / \text{SPY_return}$) from `closed_trades.db`. It flips a `FallbackFlag` with hysteresis: **enter** fallback when alpha $< 1.5\times$ SPY today; **exit** when alpha $> 1.65\times$ today AND $> 1.5\times$ for 5 sustained trading days.

`HoldSpyCoordinatorRole` at 15:55 ET runs the five-day exit/reverse transition idempotently — sells 1/5 of active positions per day and rebuys SPY on the way in, flips back on the way out.

7. State, persistence & reporting

7.1 SQLite schema (state.db)

Single SQLAlchemy ORM in `state_db.py`, 16+ tables, WAL mode for concurrent read/write between daemon, lab and supervisor. Notable tables:

- **Heartbeat, RoleRun, RoleKpi** — daemon health and per-role telemetry.
- **EquityHighWaterMark, RegimeHistory, ConfigHistory** — account and config audit.
- **Leaderboard, EvolutionRun, CalibrationRun, PromoterHalt** — lab tables; Leaderboard stores `per_trade_predictions_json` for the calibrator.
- **FallbackFlag (append-only), HoldSpyTransitionState** — Phase 4 hold-SPY coordinator.
- **AnthropicCostLog, CostHalt, TemplateProposal** — LLM cost tracking + proposal pipeline.
- **OptionFill, OptionIvHistory, WheelCycle, WheelUniverseCache** — Phase 5 wheel state.
- **SectorCache** — yfinance-backed sector classifier with 14 d TTL.
- **EmailsSent, AlertsPending, AlertsSent** — idempotent alert pipeline.

7.2 Alembic migrations (12 total)

#	Name	Adds
001	initial_schema	config_history, equity_high_water_mark, heartbeats, regime_history, role_kpis, role_runs
002	leaderboard_and_evolution_runs	Lab primary tables
003	calibration_runs	calibration_runs, promoter_halts
004	fallback_flags	fallback_flags, hold_spy_transitions
005	anthropic_and_proposals	anthropic_cost_log, cost_halts, template_proposals
006	emails_sent	emails_sent audit table
007	trade_journal_unique_order_id	UNIQUE(entry_order_id) + dedup cleanup
008	lab_promotions	first-24h validation tracking
009	schedule_audits	cron-fire audit table
010	alerts_pending	alerts_pending, alerts_sent, bot_meta
011	wheel_strategy	option_fills, option_iv_history, wheel_cycles, wheel_universe_cache
012	sector_cache	sector_cache (yfinance + ETF static map)

7.3 Runtime files (data/)

- `heartbeat.json` — `{ts, pid, version, last_action}`; written every 60 s via `tmp+rename`.
- `last_scan.json` — most-recent scan's regime, universe, per-symbol decisions.
- `portfolio_snapshot.json` — equity + positions, written by `PortfolioMonitorRole`.

- `paper_active.json + .pre-promote-backup` — active config, replaced atomically by promoter.
- `vip_seen.json` — last-seen Truth Social post ids per handle (idempotency).
- Other DBs: `backtest_bars.db`, `backtest_trades.db`, `closed_trades.db`, `massive_grouped.db`, `news_sentiment.db`, `trade_journal.db`.

7.4 Reporting & email

Every email is built from a single light-first / dark-mode-override shell in `email_shell.py`. The recent commits `fe1e797` ('every email dark-themed + EOD session review section') and `da1048e` ('dark-mode-native overhaul + daily digest 16:30 ET') confirm the current cadence.

Email	Cadence	Builder	Highlights
Midday Snapshot	12:00 ET weekdays	<code>email_midday.py</code>	KPI grid, today's trades, open positions, watchlist signals, opti
Daily Digest	16:30 ET weekdays	<code>email_digest.py:184-454</code>	13 sections: KPI grid, EOD Session Review, equity sparkline
Strategy Promotion	On every promotion	<code>email_promotion.py</code>	Params diff + risk-caps diff with green/red arrows, ‘Watch first
Per-trade Fill	On every fill	<code>email_fill.py</code>	Equity vs option branches; subjects: <i>STOP HIT AAPL ...</i>, <i>B
Critical Alert	On stall / drawdown breach	<code>email_critical.py</code>	LOW/MEDIUM/HIGH/CRITICAL color rails; throttled to once per hour

7.5 EOD Session Review

`session_summary.review_session(ctx)` is rule-based, no LLM round-trip, and emits a three-column "What went well / what went wrong / what could be better" block in the daily digest. Severity rail color depends on the count of 'wrong' items (good / warn / bad). Checks include: P&L magnitude, risk-cap proximity (75% of cap → warn), 7-day closed-trade win rate ($\geq 60\%$ well, $\leq 40\%$ wrong), errors, daemon blips, schedule audit warnings, regime context, VIX context (above threshold → warn; below 13 → wheel premiums thin), wheel MTD P&L, wheel collateral approaching cap ($> 18\%$), pending strategy promotions. Always inserts at least one positive observation as a safety net.

7.6 Dashboard

`src/trading_bot/dashboard/app.py` is a FastAPI app with a TTL-driven snapshot cache and a background refresher thread. Routes: `index`, `architecture`, `refresh`, HTMX fragment routes, and JSON APIs `/api/snapshot`, `/api/equity_curve`, `/api/market_session`. `_lab_views()` reuses `lab_data.py` so the dashboard and digest show identical numbers — single source of truth.

8. Schedule timetable & roles

8.1 Daemon job timetable (America/New_York)

Time	Job	Cadence	Role
Continuous	heartbeat	60 s interval	HealthPulseRole
06:00 weekdays	strategy_coach	Cron daily	StrategyCoachRole
06:30 weekdays	massive_refresh	Cron daily	UniverseCuratorRole.run_refresh
07:30 weekdays	premarket_rank	Cron daily	UniverseCuratorRole.run_rank
08:55 weekdays	news_warm_morning	Cron daily	SentimentAnalystRole
09:00–15:00	intel_scan (stocks)	Hourly weekdays	StockScannerRole
09:00–15:00	portfolio_watch	Hourly weekdays	PortfolioMonitorRole
09:30 onwards	crypto_scanner	Every 30 min, 24/7	CryptoScannerRole
09:45 weekdays	iv_capture	Cron daily	run_iv_capture
10:15 weekdays	wheel_scan	Cron daily	run_wheel_scan
10:00–15:00	wheel_manage	Every 30 min	run_wheel_manage
12:00 weekdays	news_warm_midday	Cron daily	SentimentAnalystRole
12:00 weekdays	midday_rerank + snapshot	Cron daily	UniverseCuratorRole + cli.midday_snapshot_cli
09:00–16:00	vip_scan	Every 30 min	VipListenerRole
Every :20, :50	verify_stops	Every 30 min, 24/7	OrderStewardRole
15:55 weekdays	hold_spy_coordinator	Cron daily	HoldSpyCoordinatorRole
16:05 weekdays	reconciler	Cron daily	reconcile_cli + reconcile_options
16:30 weekdays	daily_digest	Cron daily	ReporterRole.run_eod
21:30 nightly	wheel_universe_build	Cron daily	run_universe_build
21:55 nightly	reconciler	Cron daily	reconcile_cli (final)
03:00 Sunday	log_rotation	Weekly	LogRotationRole

8.2 The role roster (23 roles)

Role file	Tier	Process	Purpose
health_pulse.py	—daemon		Heartbeat writer (60 s); writes heartbeat.json for stall detection
sentiment_analyst.py	1	daemon	Polygon news + sentiment warm-cache (08:55, 12:00 ET) + on-demand
universe_curator.py	1	daemon	Maintain tradable list; massive_refresh + rank stage 1/2
vip_listener.py	1	daemon	Poll Truth Social RSS every 30 min RTH; flag HIGH-severity posts
stock_scanner.py	2	daemon	Hourly equity momentum scan (intel-scan); BUY/HOLD/SKIP

Role file	Tier	Process	Purpose
crypto_scanner.py	2	daemon	30-min crypto momentum scan, 24/7; no sentiment filter
strategy_coach.py	2	daemon	Daily 30 d alpha vs SPY; flip fallback flag with hysteresis
order_steward.py	3	daemon	Verify open positions have live stops; cancel stale limit orders
hold_spy_coordinator.py	4	daemon	5-day exit/reverse transition when fallback flag flips
portfolio_monitor.py	4	daemon	60-min position snapshots + alerts during RTH
account_sentinel.py	6	supervisor	Reconcile Alpaca account; write pause.flag on drawdown breach
watchdog.py	6	supervisor	Detect daemon stall via heartbeat staleness; kickstart if needed
resource_guardian.py	6	supervisor	Disk / DB / network health monitoring
schedule_auditor.py	6	supervisor	Verify expected roles ran within grace window
reporter.py	6	daemon	Compose + send digest emails (midday 12:00, EOD 16:30)
strategy_architect.py	5	lab	Claude API integration for strategy ideation (Saturday)
code_reviewer.py	5	lab	AST validation + sandbox runtime for proposed templates
backtest_engineer.py	5	lab	Walk-forward testing + optimisation
param_optimizer.py	5	lab	Optuna-based parameter tuning
calibrator.py	5	lab	Predicted-vs-realised drift detection (Spearman)
promoter.py	5	lab	Promote promising strategy variants (auto + manual)
runner.py	—	—	BaseRole with safe_run + structured error handling
base.py	—	—	Role dataclasses (RoleResult, ReportCard, Health, HealthStatus)

9. Configuration reference — strategy/config.yaml

9.1 Risk block

```
risk:
daily_loss_limit_pct: 2.0 # halt trading if daily P&L < -2%
weekly_loss_limit_pct: 5.0 # halt trading if weekly P&L < -5%
per_trade_risk_pct: 1.0 # max account at risk per trade
max_position_pct: 10.0 # no single position > 10%
max_symbol_concentration_pct: 5.0 # max 5% in any one symbol
max_consecutive_losing_days: 3 # 3 losers -> reduce sizing 50%
unprotected_stop_pct: 0.05 # auto-stop for unprotected positions
```

9.2 Allocation & regime allocations

```
allocation:
stocks_max_pct: 70.0
crypto_max_pct: 30.0
options_max_pct: 20.0
cash_floor_pct: 10.0

regime_allocations:
trending_up: {stocks: 60.0, crypto: 25.0, options: 15.0, cash: 0.0}
trending_down: {stocks: 30.0, crypto: 15.0, options: 10.0, cash: 45.0}
sideways: {stocks: 40.0, crypto: 20.0, options: 20.0, cash: 20.0}
risk_off: {stocks: 10.0, crypto: 5.0, options: 0.0, cash: 85.0}
```

9.3 Regime & sentiment

```
regime:
vol_threshold_pct: 22.0 # Phase 0b: was 30; trigger risk_off above

strategy:
sentiment_floor: -0.5 # Polygon news -1..+1; null disables filter
sentiment_max_age_days: 3
```

9.4 Wheel block

```
wheel:
enabled: true # paper-only; enabled 2026-04-29
delta_target_low: 0.20
delta_target_high: 0.30
dte_min: 30
dte_max: 45
take_profit_pct: 0.50
dte_force_close: 21
delta_breach_csp: 0.45
delta_breach_cc: 0.55
max_rolls_per_cycle: 2
iv_rank_floor: 30.0
vix_floor: 15.0
vix_ceiling: 30.0
sentiment_floor: -0.3
min_premium_abs: 0.20
min_annualized_yield: 0.12
min_open_interest: 100
universe_cache_hours: 24
wsb_spike_multiplier: 2.0
```

9.5 Email & storage

```
email:
to: bharath8887@gmail.com
daily_summary_time_et: "16:30"
weekly_summary_day: "Sunday"

storage:
trade_journal_path: data/trade_journal.db
```

Appendix — design principles

- **Soft-fail intel:** any external-source failure (timeout, 4xx, parse error) lets the entry through — gates can only veto, never trigger entries.
- **Filter-only gates:** nothing in the gate pipeline opens a position that wasn't already signalled. Adding a gate is strictly safer.
- **Per-symbol idempotency:** traded-today journal check prevents duplicates on restart (root cause of 2026-04-27 AMD/CLS/AMD incident).
- **VIP alerts are human-mediated:** Truth Social monitoring never auto-trades, auto-halts or auto-vetoes — backtest validation required first.
- **Stricter live caps:** live promotion locks max_position 5% (vs 10%), daily_loss 1.5% (vs 2%), max_drawdown 10% (vs 20%).
- **WAL mode** on state.db so daemon, lab and supervisor read concurrently without deadlocks.
- **Tmp+rename** on heartbeat, paper_active.json, alerts — atomic file replacement prevents partial-write reads.
- **Single source of truth** for lab metrics: lab_data.py feeds both the dashboard and the daily digest.